

02-prometheus-architecture

Prometheus Architecture

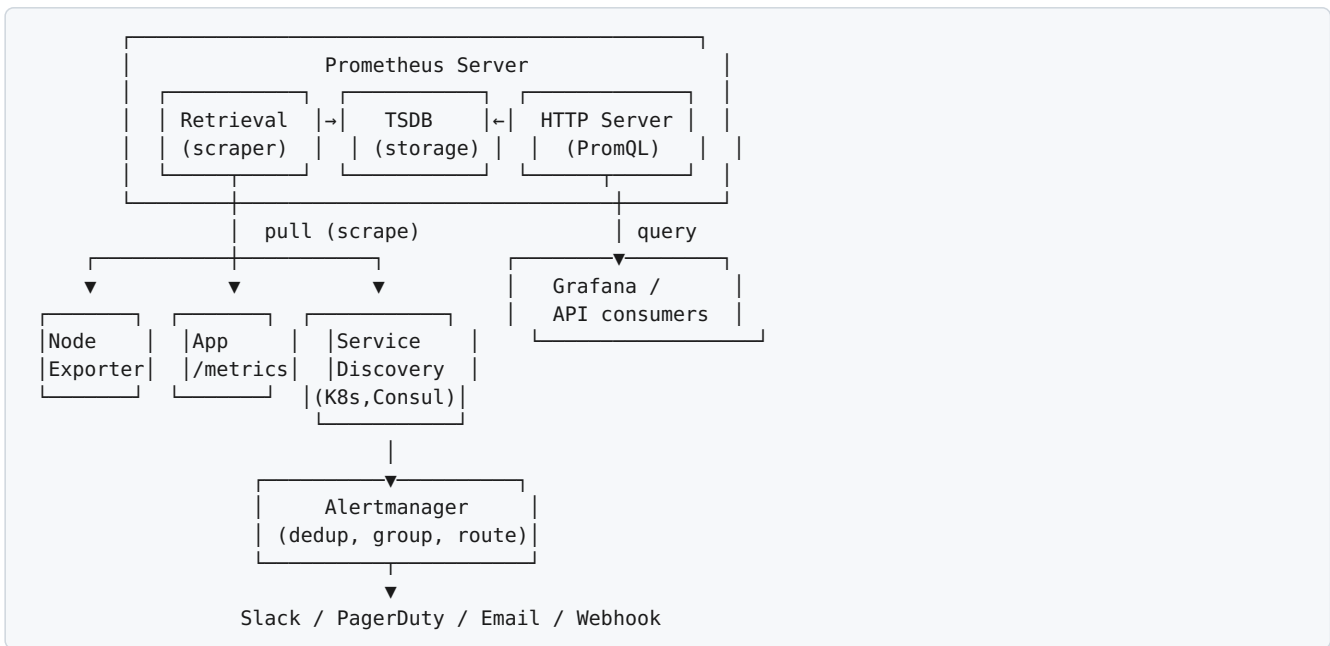
1. Overview
2. Core Architecture Components
 - 2.1 Prometheus Server
 - 2.2 Exporters
 - 2.3 Service Discovery
 - 2.4 Pushgateway
 - 2.5 Alertmanager
 - 2.6 PromQL
3. The Pull Model
 - Why pull?
 - Trade-offs
4. Data Model
 - Metric Types
5. Storage & Scalability
6. High Availability Pattern
7. Summary
- Lab: Standing up a Prometheus Stack from Scratch
 - Prerequisites
 - Step 1 — Directory setup
 - Step 2 — docker-compose.yml
 - Step 3 — prometheus.yml
 - Step 4 — alert_rules.yml
 - Step 5 — alertmanager.yml
 - Step 6 — Launch and verify
 - Step 7 — Trigger the alert
 - Step 8 — Cleanup
 - Reflection

Prometheus Architecture

1. Overview

Prometheus is an open-source systems monitoring and alerting toolkit, originally built at SoundCloud, now a graduated CNCF project. It is built around a **time-series database (TSDB)** and a **pull-based** metrics collection model.

2. Core Architecture Components



2.1 Prometheus Server

The core component with three responsibilities: - **Retrieval (scraper)**: pulls metrics from targets over HTTP at a configured interval - **TSD B (storage)**: stores time-series data locally on disk (custom format, chunks + WAL) - **HTTP server**: exposes the `/api/v1/query` (PromQL) endpoint and web UI

2.2 Exporters

Small processes that expose metrics in Prometheus's text-based exposition format at a `/metrics` HTTP endpoint. Prometheus scrapes these.

Common exporters: | Exporter | Purpose | |---| | `node_exporter` | Host-level metrics (CPU, memory, disk, network) | | `kube-state-metrics` | Kubernetes object state (pods, deployments, etc.) | | `cAdvisor` | Per-container resource usage (built into kubelet) | | `blackbox_exporter` | Probing endpoints over HTTP/TCP/ICMP/DNS | | `mysqld_exporter`, `redis_exporter`, etc. | Database/service-specific metrics |

Some applications expose metrics natively (instrumented via a Prometheus client library) instead of needing a separate exporter.

2.3 Service Discovery

Instead of statically listing every target, Prometheus can dynamically discover targets via: - Kubernetes API (pods, services, endpoints) - Consul, EC2, Azure, GCE - File-based SD (a JSON/YAML file updated externally)

This is essential in dynamic environments like Kubernetes where pod IPs change constantly.

2.4 Pushgateway

For batch/short-lived jobs that don't live long enough to be scraped (e.g., a cron job), the job can **push** its metrics to the Pushgateway, which Prometheus then scrapes like a normal target. Used sparingly — it's an exception to the pull model, not a replacement.

2.5 Alertmanager

A separate component that receives alerts from the Prometheus server (defined via **alerting rules** in PromQL) and handles: - **Deduplication** — collapsing repeated alerts - **Grouping** — batching related alerts into one notification - **Routing** — sending different alerts to different receivers based on labels - **Silencing/Inhibition** — suppressing alerts during maintenance or when a higher-severity alert already fired

2.6 PromQL

Prometheus's functional query language for slicing and aggregating time-series data. Examples:

```
# Request rate over 5 minutes
rate(http_requests_total[5m])

# 95th percentile latency
histogram_quantile(0.95, rate(http_request_duration_seconds_bucket[5m]))

# Alert condition: high memory usage
(node_memory_MemTotal_bytes - node_memory_MemAvailable_bytes) / node_memory_MemTotal_bytes > 0.9
```

3. The Pull Model

Prometheus's defining architectural choice is **pulling** metrics (scraping) rather than having applications **push** metrics to a central collector (as in StatsD/Graphite).

Why pull?

- Prometheus can detect if a target is down (a failed scrape is itself signal)
- Centralized control over scrape intervals and target lists
- Easier to run locally/test a target by simply hitting its `/metrics` endpoint
- Avoids a "thundering herd" of pushes overwhelming a central collector

Trade-offs

- Doesn't work well for very short-lived/ephemeral jobs (solved via Pushgateway)
- Requires network reachability from Prometheus to every target (can complicate firewalled/NAT'd environments)

4. Data Model

Prometheus stores everything as **time series** identified by a metric name and a set of key-value **labels**:

```
http_requests_total{method="GET", status="200", handler="/api/v1/users"}
```

Each unique combination of label values is a distinct time series. This is powerful but a source of a common pitfall: **cardinality explosion** — adding a high-cardinality label (e.g., user ID) can create millions of series and blow up memory/storage usage.

Metric Types

Type	Description	Example
Counter	Monotonically increasing value	<code>http_requests_total</code>
Gauge	Value that can go up or down	<code>node_memory_usage_bytes</code>
Histogram	Samples observations into configurable buckets	<code>http_request_duration_seconds</code>
Summary	Similar to histogram but calculates quantiles client-side	<code>rpc_duration_seconds</code>

5. Storage & Scalability

- Local TSDB stores data in 2-hour blocks on disk, with a write-ahead log (WAL) for durability
- Default retention is often 15 days locally
- For long-term storage and horizontal scale, Prometheus integrates with **remote_write** to systems like:
 - **Thanos** — adds global query view, long-term object storage (S3/GCS), downsampling
 - **Cortex / Mimir** — horizontally scalable, multi-tenant Prometheus-compatible storage
 - **VictoriaMetrics** — high-performance alternative storage backend

6. High Availability Pattern

Prometheus itself is typically run as **two identical replicas** scraping the same targets independently (since it's not

natively clustered). Deduplication happens downstream (e.g., in Thanos Query or Alertmanager's own HA clustering via gossip protocol).

7. Summary

- Pull-based scraping from `/metrics` HTTP endpoints
- Exporters bridge non-instrumented systems into Prometheus's format
- Service discovery makes it dynamic-environment friendly (esp. Kubernetes)
- PromQL is the query language for both dashboards and alerting rules
- Alertmanager is a separate, decoupled component for notification routing
- Scale beyond a single node via `remote_write` to Thanos/Cortex/Mimir

Lab: Standing up a Prometheus Stack from Scratch

Goal: Deploy Prometheus, scrape a Node Exporter, write a PromQL query, and configure a basic alert routed through Alertmanager.

Prerequisites

- Docker & Docker Compose

Step 1 — Directory setup

```
mkdir prometheus-lab && cd prometheus-lab
```

Step 2 — docker-compose.yml

```
version: "3.8"
services:
  node_exporter:
    image: prom/node-exporter
    ports:
      - "9100:9100"

  prometheus:
    image: prom/prometheus
    ports:
      - "9090:9090"
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml
      - ./alert_rules.yml:/etc/prometheus/alert_rules.yml
    command:
      - '--config.file=/etc/prometheus/prometheus.yml'

  alertmanager:
    image: prom/alertmanager
    ports:
      - "9093:9093"
    volumes:
      - ./alertmanager.yml:/etc/alertmanager/alertmanager.yml
```

Step 3 — prometheus.yml

```

global:
  scrape_interval: 10s

alerting:
  alertmanagers:
    - static_configs:
      - targets: ['alertmanager:9093']

rule_files:
  - "alert_rules.yml"

scrape_configs:
  - job_name: 'node'
    static_configs:
      - targets: ['node_exporter:9100']
  - job_name: 'prometheus'
    static_configs:
      - targets: ['localhost:9090']

```

Step 4 — alert_rules.yml

```

groups:
  - name: basic-alerts
    rules:
      - alert: HighCPULoad
        expr: node_load1 > 1.5
        for: 30s
        labels:
          severity: warning
        annotations:
          summary: "High CPU load on {{ $labels.instance }}"
          description: "1-minute load average is {{ $value }}"

```

Step 5 — alertmanager.yml

```

route:
  receiver: 'default'
receivers:
  - name: 'default'
    webhook_configs:
      - url: 'http://localhost:5001/alert'
        send_resolved: true

```

Step 6 — Launch and verify

```
docker compose up -d
```

- Visit `http://localhost:9090/targets` — confirm both `node` and `prometheus` targets are `UP`
- Visit `http://localhost:9090/graph`, run:

```
node_load1
```

- Visit `http://localhost:9090/alerts` — confirm `HighCPULoad` rule is loaded
- Visit `http://localhost:9093` — Alertmanager UI

Step 7 — Trigger the alert

Generate CPU load inside a container to push `node_load1` above 1.5 (e.g., run `yes > /dev/null &` a few times inside a stress container attached to the same host), then watch the alert transition from **inactive** → **pending** → **firing** in the Prometheus `/alerts` page and confirm it appears in Alertmanager.

Step 8 — Cleanup

```
docker compose down -v
```

Reflection

1. What happens in the Prometheus UI if you stop `node_exporter` ? How is target-down state itself observable?
2. Why does the alert rule use `for: 30s` instead of firing instantly?
3. What would change if `node_load1` had a very high-cardinality label attached to it?